PYTHON CONCURRENCY

GIL у Python

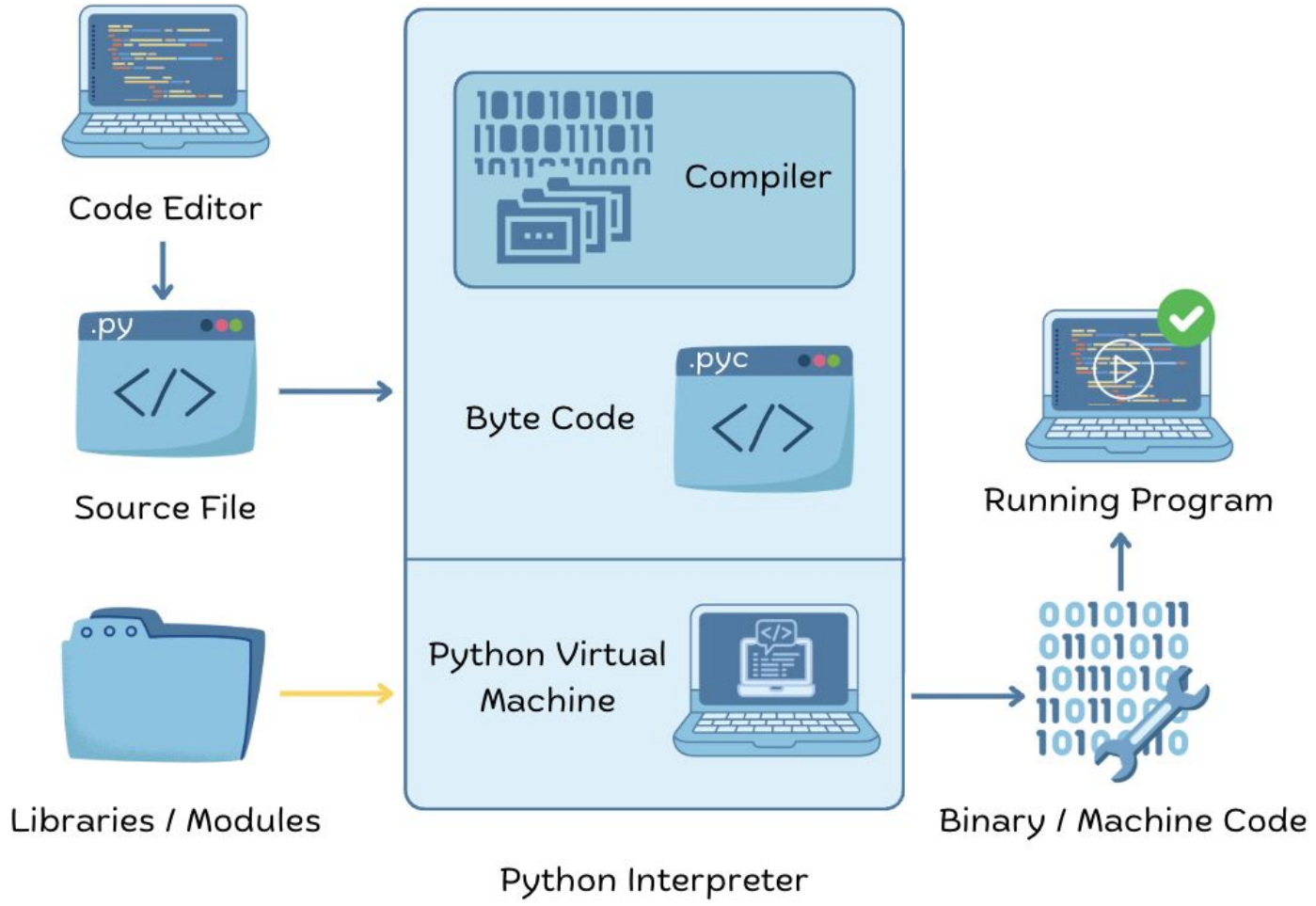
Ключ до стабільності чи ворог продуктивності?

Технічний розбір механізму Global Interpreter Lock, його вплив на багатопотоковість та майбутнє без GIL у Python

Сумець Світлана

2026

CPython інтерпретатор



Що таке GIL?

> Global Interpreter Lock - це мьютекс, який дозволяє лише одному потоку керувати інтерпретатором Python одночасно.

Ексклюзивний доступ

Гарантує, що в будь-який момент часу байт-код виконується лише одним потоком, навіть на багатоядерних CPU.

Специфіка CPython

Це особливість стандартної реалізації (CPython). Інші реалізації (Jython, IronPython) GIL не мають.

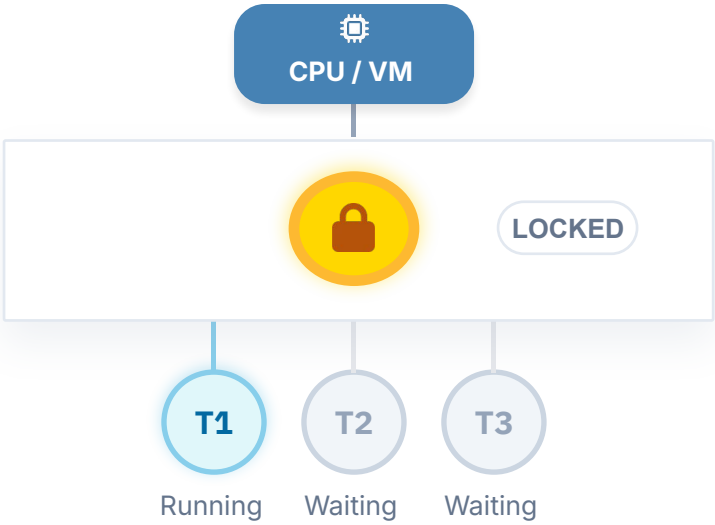
Обмеження паралелізму

Багатопотокові програми на CPU-bound задачах фактично працюють послідовно (concurrently), а не паралельно (parallel).

Безпека пам'яті

Захищає внутрішні структури даних CPython від стану гонки (race condition) та спрощує інтеграцію C-бібліотек.

МЕХАНІЗМ БЛОКУВАННЯ



Аналогія: Це як кімната з одними дверима та одним ключем. Навіть якщо людей (потоків) багато, увійти може тільки той, хто має ключ.

Чому існує GIL?

> GIL був створений не як помилка, а як необхідне інженерне рішення для архітектури управління пам'яттю Python.

Reference Counting

CPython використовує підрахунок посилань для управління пам'яттю. Змінна лічильника повинна бути захищена від одночасних змін.

Захист від гонок потоків

Без GIL два потоки могли б одночасно зменшити лічильник посилань, що призвело б до передчасного видалення об'єкта.

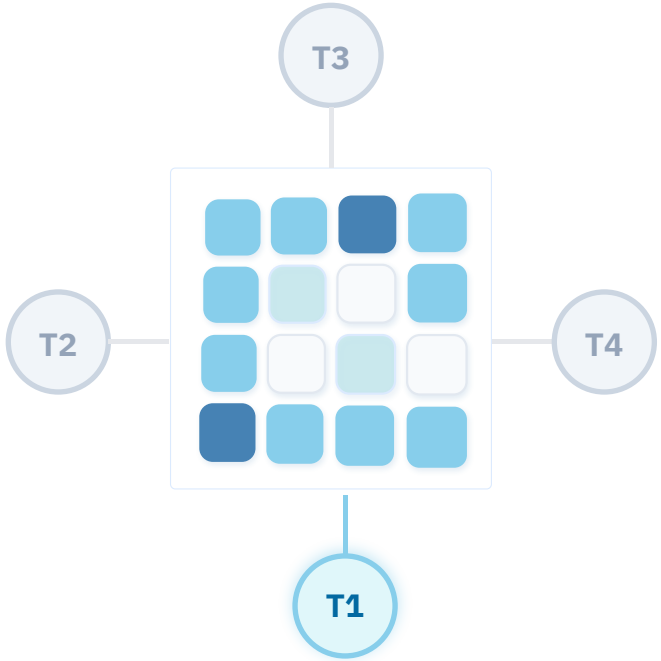
Простота C-розширень

Багато бібліотек на C (NumPy, SciPy) покладаються на GIL для безпечної роботи з об'єктами Python без складних блокувань.

Продуктивність Single-Thread

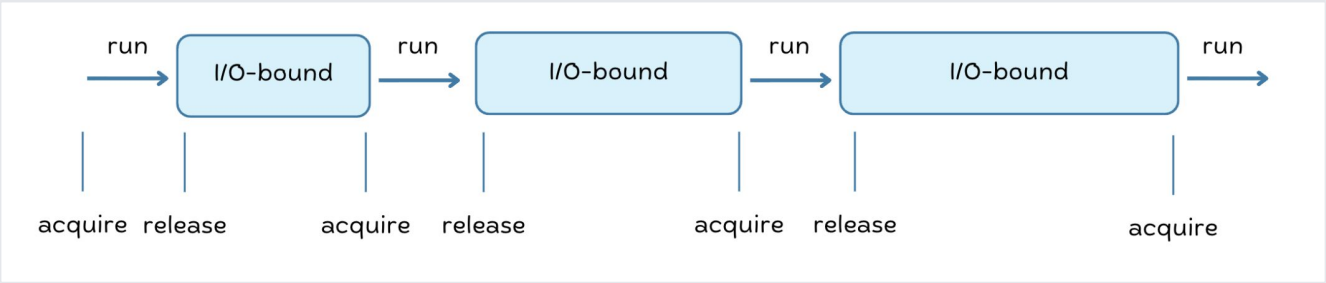
Відсутність дрібних блокувань (fine-grained locking) робить однопоточні програми значно швидшими.

ЗАХИСТ ПАМ'ЯТІ



Дилема: Без GIL довелося б ставити блокування на кожну операцію з кожним об'єктом, що зробило б Python повільним і складним.

Цикл виконання байт-коду (eval loop)



Як це працює?

> **Нескінченний цикл**
>
> Інтерпретатор послідовно читає та виконує інструкції байт-коду.

Перевірка eval_breaker. Періодично (кожні N тіків або 5ms) перевіряється, чи потрібно зупинитись (для GC, сигналів або переключення потоків).

Виконання. Якщо прапори чисті, виконується наступна операція (наприклад, LOAD_FAST, BINARY_ADD).

```
for (;;) {
   // Перевірка необхідності зупинки
   if (_Py_atomic_load_relaxed(eval_breaker)) {
      if (eval_frame_handle_pending(tstate) != 0)
         goto error; // Release GIL here
   }

   // Отримання та виконання інструкції
   NEXTOPARG();
   switch (opcode) {

   }
}
```

CPython implimentation

I/O-Bound: Результати аналізу

БЛОКУВАННЯ

GIL Usage

0.00%

✓ Released

Статус	Вільний
Конкуренція	Відсутня
Причина	time.sleep / I/O

Потоки автоматично звільняють GIL під час операцій вводу/виводу

АКТИВНІСТЬ CPU

Active Time

24.00%

Waiting

Idle Time	76%
Overhead	Низький
Resource	Network / Disk

Процесор не навантажений очікуванням, ресурси використовуються ефективно

ПАРАЛЕЛІЗМ

Total Threads

5 Threads

Concurrent

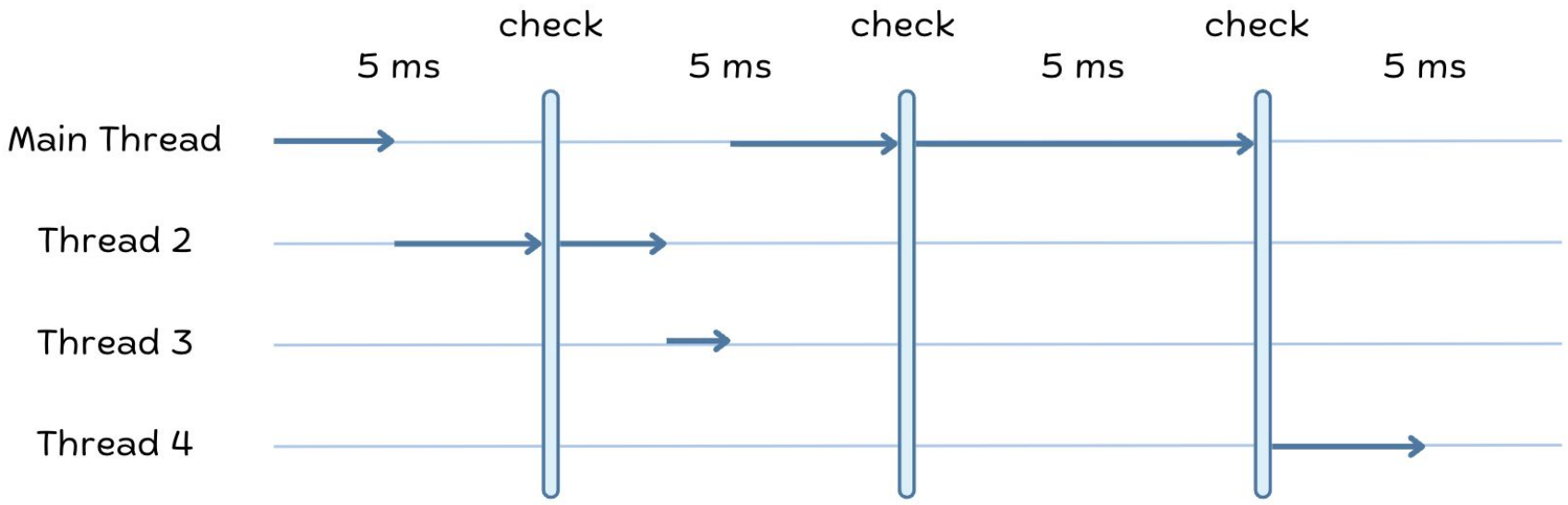
Main Thread	1 (Management)
Worker Threads	4 (Tasks)
Efficiency	Висока

Всі 5 потоків працюють без взаємного блокування GIL

Для I/O-bound задач Python-потоки є відмінним інструментом, оскільки GIL не стає вузьким місцем.

Source: py-spy profile data

Перемикання потоків



Механізм "eval_breaker"

Інтерпретатор перевіряє змінну eval_breaker кожні N байт-код інструкцій. Якщо час вийшов (5ms), поточний потік змушений "скинути" GIL, даючи шанс іншим.

Switch Interval

За замовчуванням: 5ms

До Python 3.2 використовували "тіки" (100 інструкцій), зараз - таймер.

Баланс

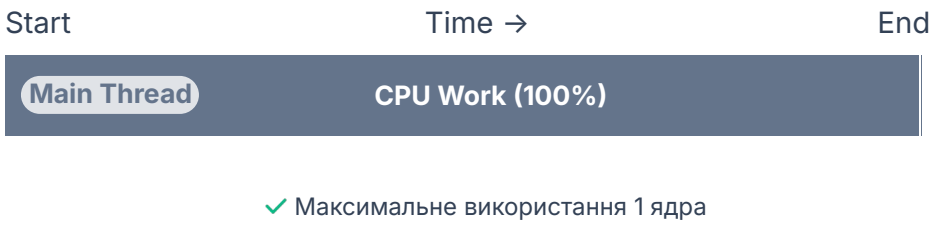
Малий інтервал = високий overhead.
Великий інтервал = довгий відгук (lag).

CPU-bound задачі: Обмеження

Один потік

Baseline

Послідовне виконання на 1 ядрі



✓ Максимальне використання 1 ядра

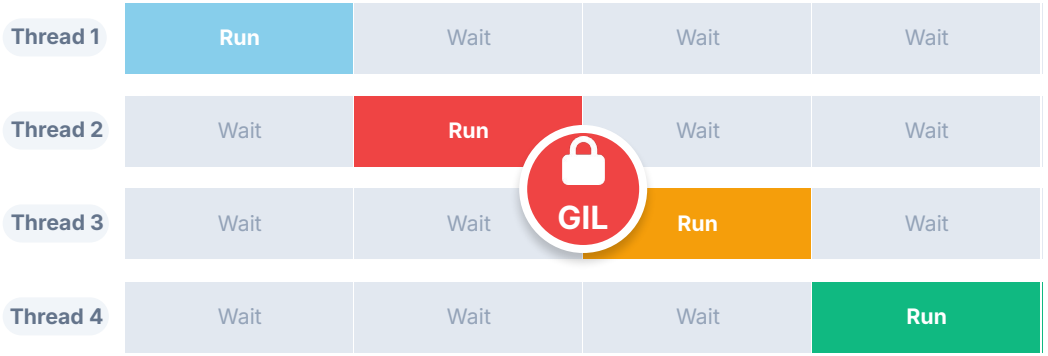
Час виконання	15.0 сек
Завантаження CPU	100% (1 core)
Накладні витрати	Мінімальні

VS

Багатопотоковість

GIL Limited

Конкуренція за GIL на 4 ядрах



Час виконання	~15.2 сек (Повільніше!)
Завантаження CPU	~100% (сумарно)
Накладні витрати	Context Switching

CPU-bound: Метрики py-spy

БЛОКУВАННЯ

GIL Usage

99.00%

Серійне виконання

Стан	Locked
Паралелізм	Відсутній
Утилізація ядер	1 Core Max

Лише один потік виконує Python-код у кожен момент часу

АКТИВНІСТЬ

Active Time

107.00%

Високий Overhead

Корисна робота	~95%
Context Switch	~12% (Втрати)
Ефективність	Низька

>100% означає, що ми витрачаємо ресурси на боротьбу за лок

КОНКУРЕНЦІЯ

Threads

4 Threads

Resource Contention

Start Overhead	Високий
Memory Usage	4x Stack
Результат	Повільніше за 1 потік

Додаткові потоки лише створюють чергу до GIL

Висновок: Для CPU-bound задач багатопотоковість у Python (CPython) є шкідливою. Використовуйте multiprocessing.

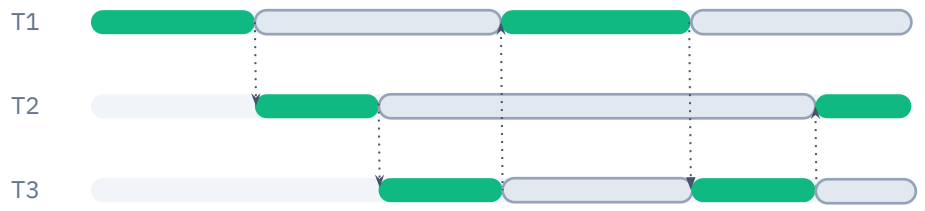
Source: py-spy top -- python CPU_task.py

I/O vs CPU: Коли потоки виграють?

I/O-Bound

Перемога потоків

Мережа, Диск, База даних



GIL звільнений (конкурентність)

Статус GIL	Released (Звільнений)
Ефект	Справжня конкурентність

Рекомендація: Використовуйте threading або asyncio. Потоки ефективні, бо час очікування I/O використовується іншими.

VS

CPU-Bound

Поразка потоків

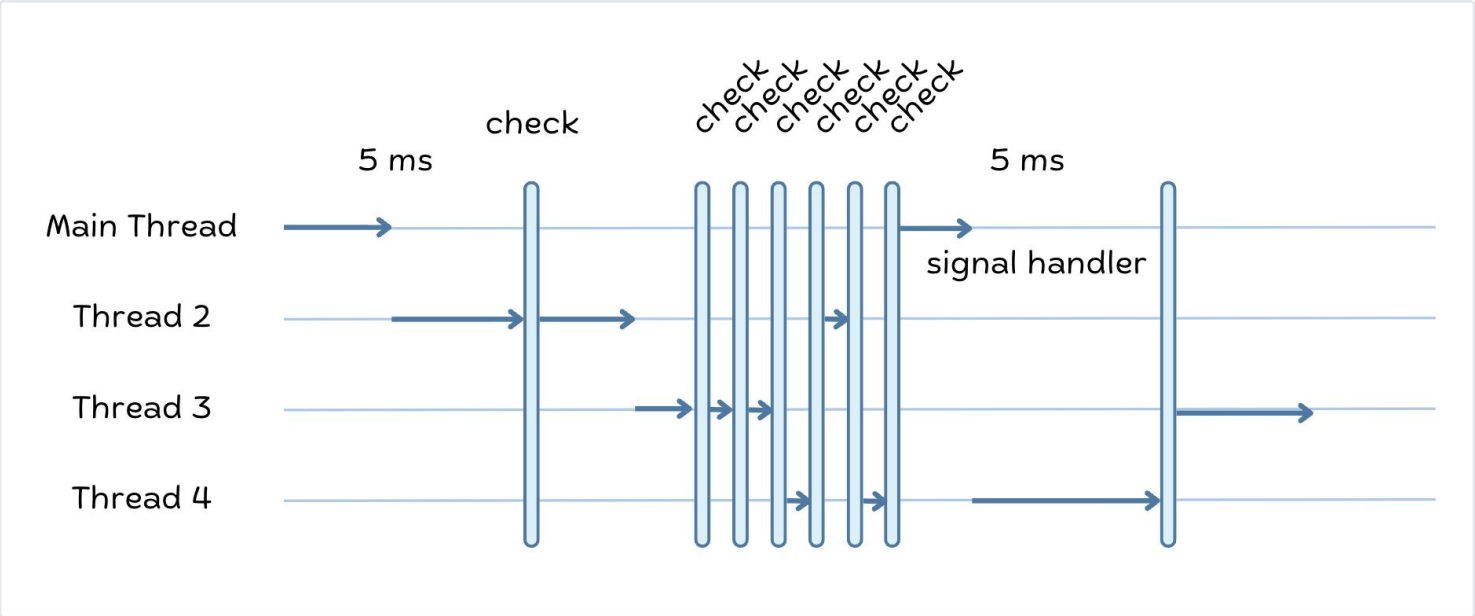
Обчислення, ML, Обробка зображень

GIL заблокований (серіалізація)

Статус GIL	Held (Утримується)
Ефект	Послідовне виконання + Overhead

Рекомендація: Використовуйте multiprocessing. Потоки лише додають накладні витрати на перемикання контексту.

Сигнали та Ctrl+C (SIGINT)

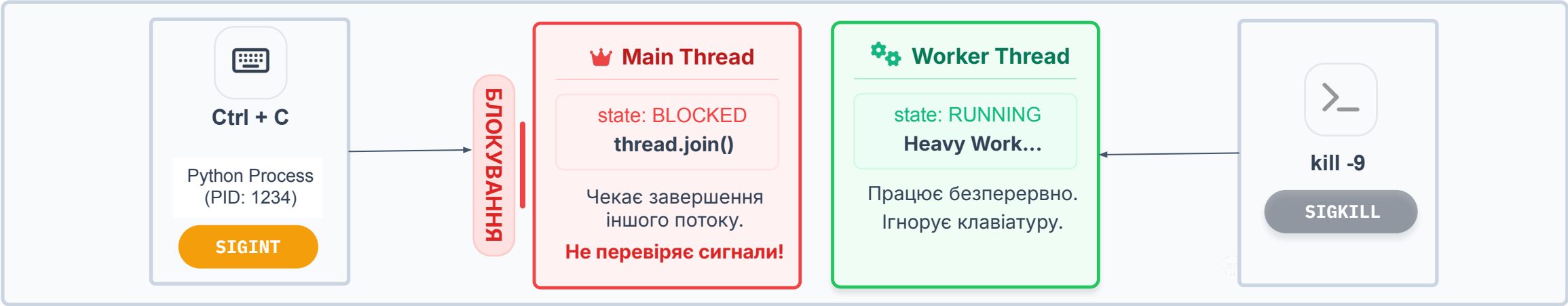


⚠ Головний потік only

У Python обробники сигналів (як KeyboardInterrupt) можуть виконуватися тільки в головному потоці. Інші потоки їх ігнорують.

🚫 Пастка блокування

Якщо Main Thread заблокований у системному виклику (наприклад, thread.join() без таймауту), він не може "прокинутися" для перевірки прапорів сигналів.



Планування потоків: роль ОС

> Python не керує тим, який саме потік отримає GIL наступним. Це повністю залежить від планувальника операційної системи.

Влада ОС

Інтерпретатор лише повідомляє ОС, що готовий віддати GIL. ОС вирішує, який з готових (ready) потоків розбудити.

Відсутність пріоритетів

У Python немає вбудованого API для задання пріоритетів потоків (на відміну від Java чи C#). Всі потоки рівні.

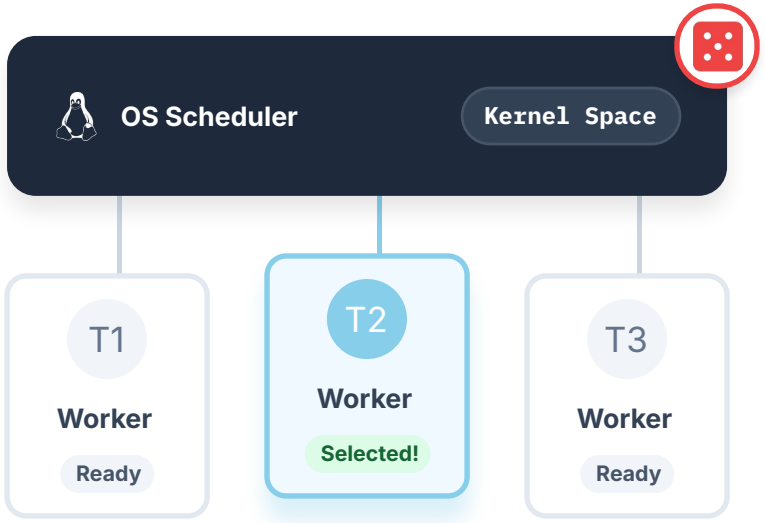
Недетермінованість

Порядок виконання не гарантується. Не можна покладатися на те, що потік A виконається раніше за потік B.

Вплив навантаження

Високе навантаження системи іншими процесами може впливати на те, як часто ваші потоки отримують час CPU.

МЕХАНІЗМ ВИБОРУ



Важливо: Ніколи не використовуйте time.sleep() для синхронізації потоків. Це "крихке" рішення через непередбачуваність планувальника.

Як реалізований GIL: примітиви

Компоненти синхронізації
OS Threads & Locking

Mutex
Взаємне виключення: гарантує, що лише один потік володіє GIL.
`pthread_mutex_t`

GIL

Condition Variable
Сигналізація: потоки чекають сигналу про звільнення GIL.
`pthread_cond_t`

Switch Variable
Контроль перемикання (switch interval, eval_breaker).
`_Py_atomic_int`

Pthreads Implementation
Залежить від ОС

Locking | Signaling | Control

Еволюція Fairness
History of Improvements

Python 2.x - 3.1

Tick-based GIL
Перемикання кожні 100 інструкцій байт-коду. Проблема "конвою" на багатоядерних CPU.

Python 3.2+

Time-based GIL
"New GIL" від Antoine Pitrou. Інтервал 5 мс. Примусове звільнення для уникнення голодування I/O потоків.

Python 3.13

No-GIL (Experimental)
PEP 703: Можливість повної відмови від GIL. Free-threaded виконання.

Fairness Improvements
Боротьба з "голодуванням" потоків

М'ютекси: Захист ресурсів

Мета та Реалізація

Концепція та Системні виклики

Головна мета

Не дозволити більше ніж одному потоку одночасно отримати доступ до спільного ресурсу. Це критично для уникнення стану гонитви (race conditions) та забезпечення коректності виконання програми.

НИЗЬКОРІВНЕВА РЕАЛІЗАЦІЯ

Unix / Linux

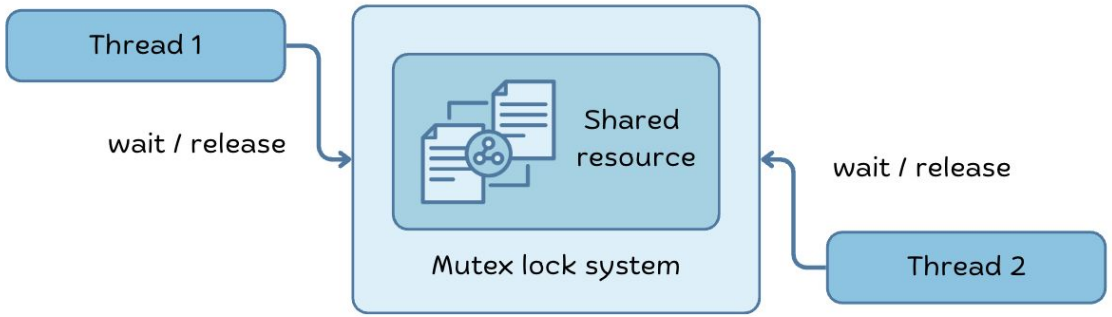
```
# POSIX Threads (pthreads)
int pthread_mutex_lock(pthread_mutex_t *mutex);
int pthread_mutex_unlock(pthread_mutex_t *mutex);
```

Windows

```
# Win32 API
void EnterCriticalSection(LPCRITICAL_SECTION lpCriticalSection);
void LeaveCriticalSection(LPCRITICAL_SECTION lpCriticalSection);
```

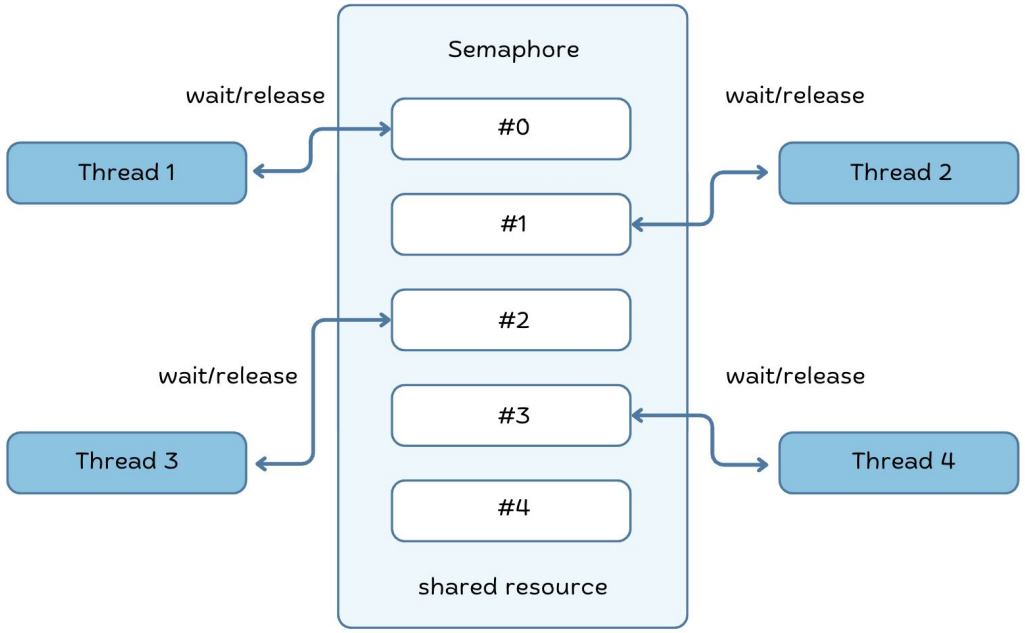
Схема Mutex Lock in OS

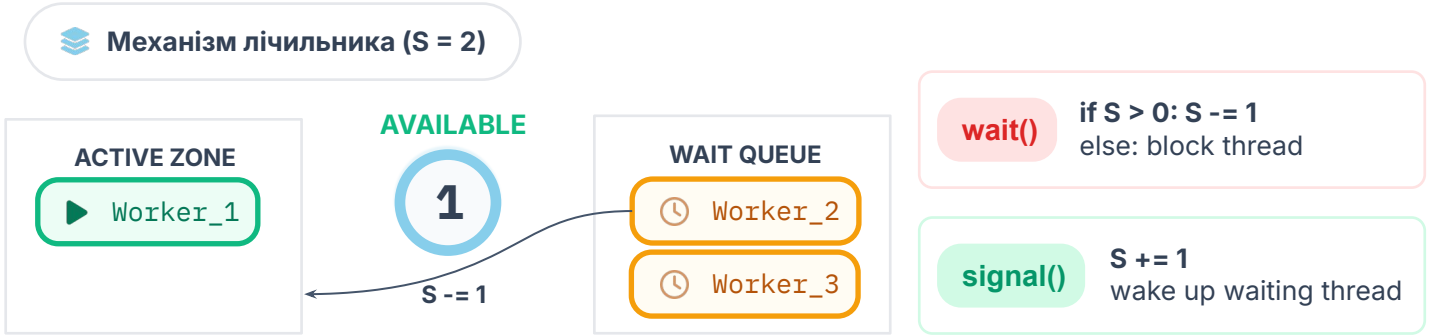
Візуалізація взаємодії потоків



Важливо: м'ютекс - це не просто логічний прапорець, а примітив синхронізації з участю ядра ОС. У разі конфлікту потік переводиться в стан очікування, що запобігає busy-waiting і зменшує навантаження на CPU. Коректне використання м'ютексів є критичним для уникнення race conditions і data corruption.

Семафори: Регулювання потоку





Що таке Семафор?

Це цілочисельна змінна, яка використовується для вирішення задач синхронізації. На відміну від м'ютекса, семафор дозволяє доступ до ресурсу N потокам одночасно.

Відмінність від М'ютекса GIL

М'ютекс: Власник – лише 1 потік. Хто заблокував, той і розблоковує.

Семафор: Сигналізувати (розблокувати) може будь-який потік. Використовується для сповіщень.

ЗАСТОСУВАННЯ

Пул з'єднань БД
Обмеження кількості активних запитів

Producer-Consumer
Координація черги завдань

Умовні змінні (Condition Variables)

Механізм роботи

Точка синхронізації: Потоки "засинають", чекаючи певної умови (наприклад, звільнення GIL).

Взаємодія з м'ютексом: pthread_cond_wait атомарно звільняє м'ютекс і блокує потік, додаючи його в чергу.

Ефективність (No Busy-Wait): Потік не витрачає цикли CPU на опитування (polling), а чекає пробудження від ядра ОС.

Таймаут: Можливість розбудити потік примусово, якщо він чекає занадто довго (уникнення зависань).

КЛЮЧОВІ ОПЕРАЦІЇ

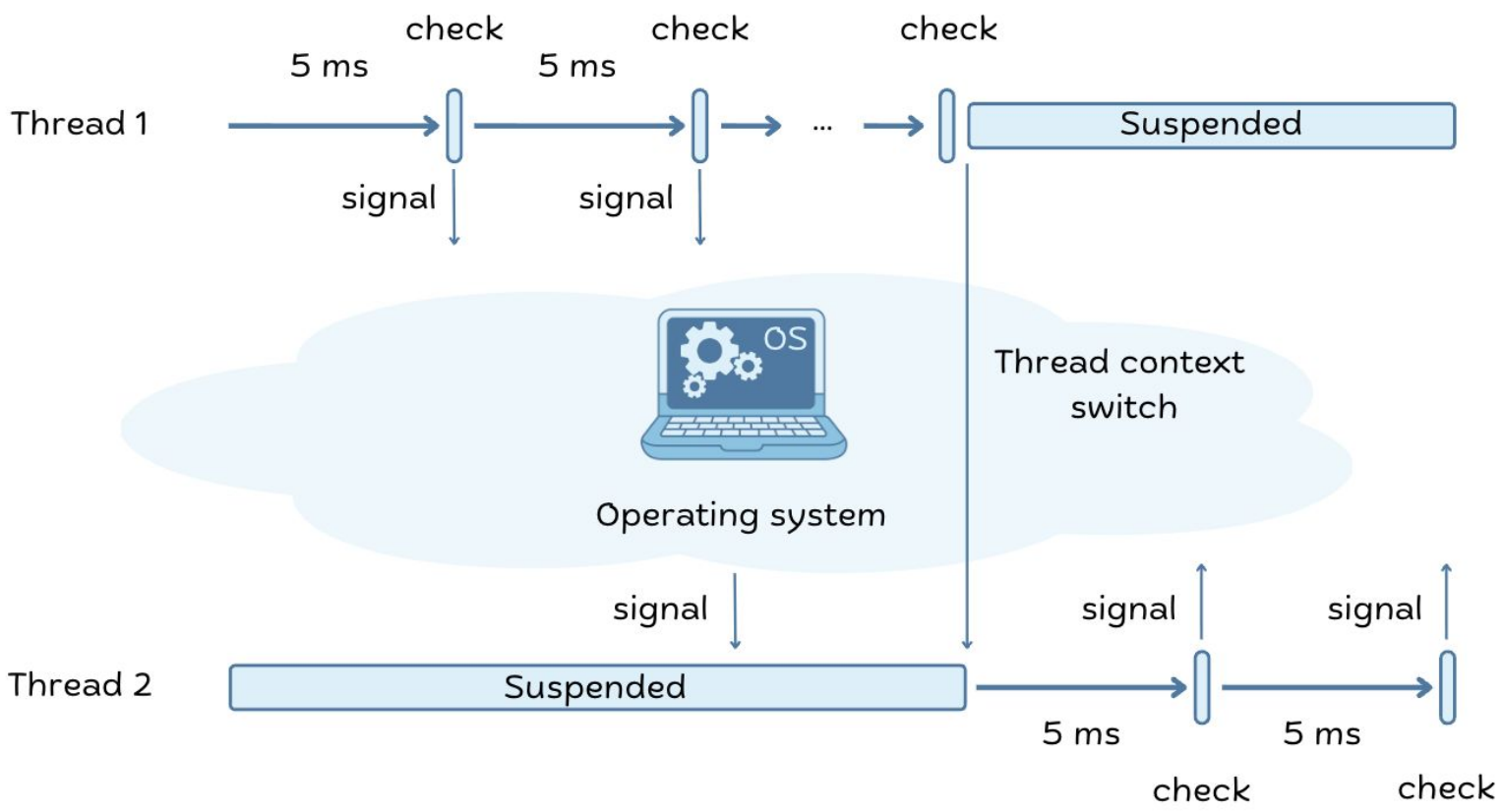
pthread_cond_wait	Блокує потік, звільняє м'ютекс. Повернення означає захоплення м'ютекса.
pthread_cond_signal	Будить один потік з черги. Використовується для передачі GIL.
pthread_cond_broadcast	Будить всі потоки. Корисно при зміні глобального стану.

Приклад: Буфер даних

D1 D2 D3 ...

Обробка починається лише коли буфер заповнений (signal від наповнювача).

Повний цикл роботи GIL



1

Захоплення

Потік намагається отримати м'ютекс GIL через pthread_mutex_lock

2

Виконання

Виконання байт-коду протягом кванту часу (~5 мс). Перевірка eval_breaker.

3

Звільнення

Потік відпускає GIL, сигналізує (signal) і переходить у стан wait (sleep).

4

Конкуренція

ОС обирає наступний потік для пробудження. Race Condition за Lock.

Недоліки GIL на практиці

> Попри стабільність, GIL стає серйозним бар'єром для високопродуктивних обчислень у сучасних багатоядерних системах.

Обмеження багатоядерності

Неможливість використання всіх ядер процесора для Python-коду. 100 потоків все одно використовують 1 ядро.

Проблеми C-розширень

Довгі обчислення в C-функціях без явного звільнення GIL блокують всі інші потоки програми ("заморожування").

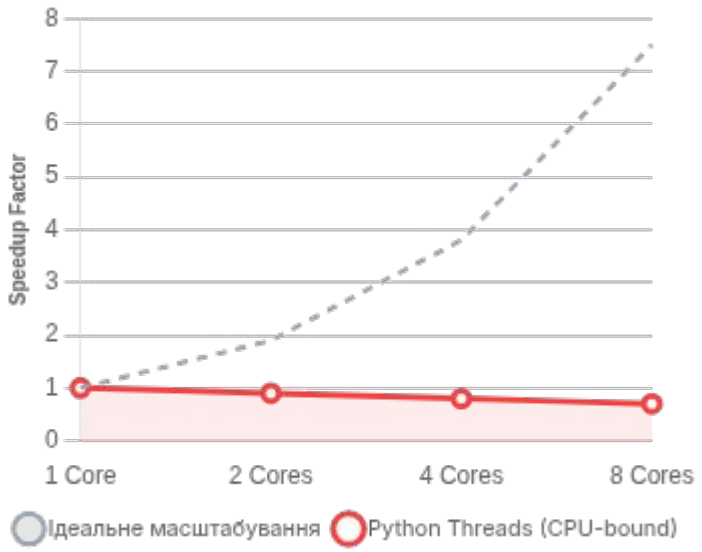
Високі накладні витрати

"Битва за GIL" (GIL contention) призводить до того, що багатопотокова програма може працювати повільніше за однопотокову.

Складність переривання

Сигнали (наприклад, SIGINT / Ctrl+C) можуть оброблятися із затримкою, якщо головний потік очікує на GIL.

МАСШТАБУВАННЯ НА ЯДРАХ



GIL Contention: При збільшенні кількості потоків на CPU-задачах продуктивність падає через витрати на перемикання контексту.

Python 3.13+: курс на free-threaded

> PEP 703 пропонує відмову від GIL, перетворюючи Python на справжню багатопотокову платформу (free-threaded).

Експериментальний режим

Доступний через прапор компіляції --disable-gil. GIL вимкнено, але сумісність не гарантована.

Справжній паралелізм

Потоки виконують байт-код одночасно на різних ядрах. Ідеально для CPU-bound задач без multiprocessing.

Ціна свободи

Однопоточний код стає повільнішим (~10-15%) через накладні витрати на атомарні операції та нове управління пам'яттю (mimalloc).

Виклики екосистеми

Бібліотеки на C (NumPy, Pandas) повинні оновити свій код, щоб забезпечити внутрішню потокобезпечність без GIL.

PEP 703 OVERVIEW — **EXPERIMENTAL**

+ ПЕРЕВАГИ

- Масштабування на ядрах
- Менше Overhead ніж у multiprocessing
- Простіша модель пам'яті

− РИЗИКИ

- Single-thread performance
- Поламка старих C-extensions
- Race conditions у user-code

Ключові тези та рекомендації

01

Потоки vs Процеси

Використовуйте threading для I/O-bound задач (мережа, диск), а multiprocessing для CPU-bound обчислень, щоб обійти обмеження GIL.

02

Профілювання

Використовуйте py-spy для аналізу. Високий % GIL означає CPU-блокування, високий % Active при низькому GIL — ефективний I/O.

03

Бібліотеки та C-API

NumPy, Pandas та Cython часто звільняють GIL під час важких обчислень. Використовуйте оптимізовані бібліотеки замість чистого Python коду.

04

PEP 703: Free-threading

Python 3.13 експериментально дозволяє роботу без GIL. Слідкуйте за оновленнями екосистеми та готуйте код до thread-safe майбутнього.

05

Кращі практики

Уникайте глобального стану, використовуйте черги (Queue) для комунікації та пам'ятайте про накладні витрати на створення потоків.

Підсумок

"GIL - це не вирок продуктивності, а інженерний компроміс. Розуміння його механізмів дозволяє писати швидкий та безпечний код."

PYTHON PERFORMANCE

Що почитати?

- CPython - ceval.c
- YouTube - Understanding the Python GIL
- PEP 703
- When Python can’t thread
- What Is the GIL?
- POSIX Threads Programming